

CHANNEL v1.2.1 (v1.2.1 Fixed) — Blocks 1 / 2 / 3

Readable PDF for Dev GPT • Generated 2026-02-06 15:37 UTC

본 문서는 채널 엔진 구현을 위한 정보(고정 규칙)입니다. 임의 변경/요약/완화/강화 금지.

BLOCK 1 — 운영 프롬프트 / 금지사항 / 완료조건

너는 지금부터 CHANNEL 멀티TF 엔진을 구현한다.

[필수]

- 블록2(정본 룰 텍스트) + 블록3(데이터모델/상태머신/커버리지)를 1:1로 반영한다.
- 규칙을 임의로 완화/강화/재해석/리팩토링 금지. 수치/상수/가드조건은 그대로 고정.
- 상태 전이는 블록3 상태머신 표에서만 발생해야 한다. 다른 곳에서 if로 state를 바꾸지 마라.
- pivot은 fractal pivotLen=3 확정 pivot만 사용한다. 확정 전 pivot은 구조 판단(BOS/CHOC)에 절대 사용 금지.
- D1/H4는 상시 운영(생성/갱신/무효화 확정), H1/M30은 상시 계산/표시하되 상위 POI 유무에 따라 ENABLED/CONTEXT_ONLY로 구분한다.
- 15m/5m는 채널 생성/갱신 금지. 오직 gate 계산에만 사용한다.

[구현 순서 고정]

- 1) 상수/enum 정의
- 2) 데이터 모델 타입 정의
- 3) 단일 업데이트 루프 onBarClose(tf) 구현 (TF별 독립 트리거)
- 4) TF별 순서 구현:
 - updateAtr(tf)
 - updatePivots(tf)
 - rebuildOrUpdateChannel(tf) (생성/재생성/종료)
 - evaluateBreakAndAnchorInvalid(tf)
 - applyTtlAndParentPoiRules(tf) (H1/M30)
 - evaluateChannelPoi(tf) (POI 조건 충족 시 생성)
 - emitEvents() (포맷 고정)
- 5) 최소 테스트 시나리오 15개 이상 작성 (커버리지 매트릭스 모든 행 커버)

[완료 조건]

- Rule Coverage Matrix 모든 행이 코드에서 어디에 구현됐는지 함수/파일 단위로 매핑되어야 한다.
- TF별 퍼센타일/브레이크/게이트/TTL/POI 조건이 전부 통과해야 한다.
- H1/M30 POI cap(UTC 기준 1일 2개/심볼/TF) 로직이 결정론적으로 동작해야 한다.
- 이벤트 문자열 포맷(블록3 템플릿) 그대로 출력.

BLOCK 2 — 정보 룰 텍스트 (CHANNEL v1.2.1)

(정보 룰 텍스트) CHANNEL v1.2.1

0) 목적(고정)

- Context: OB/FVG/추세선 자리 품질 태그(가산)
- Channel POI(희소): 경계 반응 확인 시 단독 POI 이벤트 생성(폭발 방지)

1) 적용 TF(고정)

- D1_CHANNEL: D1 close에서만 생성/갱신/무효화 확정
- H4_CHANNEL: H4 close에서만 생성/갱신/무효화 확정
- H1/M30_CHANNEL: 상시 계산/표시. 단, 상위 POI 없으면 CONTEXT_ONLY
- M15/M5: 채널 생성/갱신 금지, gate 계산만

2) 입력 데이터(고정)

- OHLC: 해당 TF
- pivot: fractal pivotLen=3 확정 pivot만
- ATR: 해당 TF ATR(14)

3) 재생성 트리거(고정)

- 초기 생성 / 브레이크 / 앵커 무효화 / TTL만료(H1/M30)

4) 앵커 규칙(고정)

- lookback: 300 bars(해당 TF)
- minSwing = $0.25 \times \text{ATR}$
- 상승: HL pivot low 2점 (lower anchor line)
- 하락: LH pivot high 2점 (upper anchor line)
- 회귀/최소제공 금지(직선 2점)

5) offset(고정, TF별 차등)

- residual(상승): $\text{low} - \text{anchorLinePrice}$ (양수만)
- residual(하락): $\text{anchorLinePrice} - \text{high}$ (양수만)
- residual 양수 표본 5개 미만이면 채널 생성 금지
- 퍼센타일: D1 p95 / H4 p90 / H1 p85 / M30 p80
- midline = $\text{offset} \times 0.5$

6) 브레이크/앵커무효화(고정, TF별 차등)

[브레이크: 바운더리 기반]

- D1: 2연속 증가 이탈 + 이탈폭 $\geq \text{ATR} \times 0.20$
- H4: 2연속 증가 이탈 + 이탈폭 $\geq \text{ATR} \times 0.20$
- H1: 1연속 증가 이탈 + 이탈폭 $\geq \text{ATR} \times 0.30$
- M30: 1연속 증가 이탈 + 이탈폭 $\geq \text{ATR} \times 0.35$
- > reason=break, 종료 후 즉시 재생성 시도

[앵커 무효화: anchor line 기반]

- 상승: 증가가 lower anchor line 아래로 위 브레이크 규칙(연속/폭) 만족
- 하락: 증가가 upper anchor line 위로 위 브레이크 규칙(연속/폭) 만족
- > reason=anchor_invalid, 종료 후 즉시 재생성 시도

7) 표시/투영/TTL(고정)

- 표시 시작: anchorStartTime부터

- 투영: 기본 MAX_FORWARD_BARS=300
- TTL(H1/M30만): H1=100 bars, M30=80 bars → reason=t1_expired
- parent 종료 연동(H1/M30, ENABLED 상태일 때):
참조 상위 POI가 INACTIVE/DELETED로 전환되면 즉시 종료(reason=parent_poi_ended)

8) 컨텍스트 태그(고정)

- 롱(상승): lower boundary 기준 거리
- 숏(하락): upper boundary 기준 거리
- OK: $dist \leq ATR * 0.25$
- TIGHT: $dist \leq ATR * 0.10$
- 우선순위(중복 가능): D1 → H4 → H1/M30
- H1/M30이 CONTEXT_ONLY이면 태그는 가능하되 약한 태그 구분 유지

9) H1/M30 상태 분류(고정)

- 상위 POI 0개 → CONTEXT_ONLY (POI 생성 금지)
- 상위 POI ≥ 1 개 → ENABLED (강조건 만족 시 POI 생성 가능)

10) CHANNEL_POI(고정)

[생성 TF]

- D1/H4 허용
- H1/M30도 허용(단 ENABLED + 추가 제한 필수)

[POI gate(경계 근처, TF별)]

- D1: $0.15 * ATR$
- H4: $0.12 * ATR$
- H1: $0.08 * ATR$
- M30: $0.06 * ATR$
- wickExtreme: 상승 wickLow / 하락 wickHigh

[트리거 3종]

- 1) sweep→recovery: wick 이탈 후 다음 봉 증가로 채널 안 복귀
- 2) structure: pivotLen=3 확정 pivot 기반 BOS/CHOCH가 채널 방향과 정합
- 3) displacement: $\max(\text{body3}) > ATR * 1.0$ OR $\sum(\text{body3}) > ATR * 1.8$

[확정 조건]

- D1/H4: 3개 중 2개 이상
- H1/M30: 3개 모두 필요(3/3)

[H1/M30 추가 제한]

- 채널 상태 ENABLED 필수
- 상위 POI near/inside 필수(둘 중 하나):
near: $\text{abs}(\text{wickExtreme} - \text{parentPoiBoundary}) \leq ATR(H1/M30) * 0.20$
inside: $\text{overlapRatio}(\text{parentPoiZone}, \text{boundaryZoneProxy}) \geq 0.20$
boundaryZoneProxy = [boundary - gate, boundary + gate] (gate는 TF별 POI gate)
- 1일 최대 2개/심볼/TF (UTC 00:00~23:59)

11) LTF(15m/5m) 활용(고정)

- 채널 재계산 금지
- gate: $\text{abs}(LTF \text{ wickExtreme} - \text{channel boundary}) \leq LTF_ATR * 0.20$

12) 이벤트 문자열 포맷(고정)

- time은 ISO-8601 UTC: YYYY-MM-DDTHH:mm:ssZ

- 필드 순서/구분자 고정(아래 템플릿 그대로)

```
[CHANNEL][NEW]
[CHANNEL][NEW][{D1|H4|H1|M30}][{UP|DOWN}] time={ISO} anchors={t1}@{p1},{t2}@{p2} offsetPct1={n} offset={x.xx} mid={x.xx}

[CHANNEL][MODE] (H1/M30만, 선택 출력)
[CHANNEL][MODE][{H1|M30}] time={ISO} mode={ENABLED|CONTEXT_ONLY}

[CHANNEL][INVALID]
[CHANNEL][INVALID][{D1|H4|H1|M30}][{channelId}] time={ISO} reason={break|anchor_invalid|ttl_expired|parent_poi_ended} endTime={ISO}

[CHANNEL][POI]
[CHANNEL][POI][{D1|H4|H1|M30}][{BULL|BEAR}] time={ISO} boundary={price} triggers={sweepRec|structure|disp}
```

BLOCK 3 — 데이터 모델 / 상태머신 / 커버리지 매트릭스

3.1 공통 상수 / 열거형

```

type TF = "D1" | "H4" | "H1" | "M30" | "M15" | "M5";
type DIR = "BULL" | "BEAR";
type ChannelDir = "UP" | "DOWN";

type ChannelType = "D1_CHANNEL" | "H4_CHANNEL" | "H1_CHANNEL" | "M30_CHANNEL";
type ChannelState = "ACTIVE" | "INACTIVE" | "NONE"; // NONE = 생성 실패(anchors 부족 등)
type ChannelMode = "ENABLED" | "CONTEXT_ONLY"; // H1/M30 전용. D1/H4는 항상 ENABLED 취급.

type ChannelInvalidReason = "break" | "anchor_invalid" | "ttl_expired" | "parent_poi_ended";

const LOOKBACK_BARS = 300;
const MAX_FORWARD_BARS = 300;

const MIN_SWING_ATR = 0.25; // minSwing = ATR*0.25
const MIN_RESIDUAL_SAMPLES = 5;

const BREAK_D1_CLOSES = 2, BREAK_D1_ATR = 0.20;
const BREAK_H4_CLOSES = 2, BREAK_H4_ATR = 0.20;
const BREAK_H1_CLOSES = 1, BREAK_H1_ATR = 0.30;
const BREAK_M30_CLOSES = 1, BREAK_M30_ATR = 0.35;

const OFFSET_PCTL_D1 = 95;
const OFFSET_PCTL_H4 = 90;
const OFFSET_PCTL_H1 = 85;
const OFFSET_PCTL_M30 = 80;

const POI_GATE_D1 = 0.15;
const POI_GATE_H4 = 0.12;
const POI_GATE_H1 = 0.08;
const POI_GATE_M30 = 0.06;

const PARENT_NEAR_ATR = 0.20;
const PARENT_INSIDE_RATIO = 0.20;

const POI_CAP_PER_DAY = 2; // per symbol per TF
// "1일"은 UTC 00:00~23:59 고정

```

3.2 데이터 모델(필드/타입)

```

type Timestamp = number; // ms epoch UTC
type Price = number;

interface Bar {
  tf: TF;
  openTime: Timestamp;
  closeTime: Timestamp;
  open: Price;

```

```

    high: Price;
    low: Price;
    close: Price;
}

interface AtrSnapshot {
    tf: TF;
    time: Timestamp; // closeTime
    atr14: number;
}

interface Zone {
    bottom: Price;
    top: Price;
    height: number; // top-bottom
}

type PivotType = "HIGH" | "LOW";
interface Pivot {
    tf: TF;
    pivotType: PivotType;
    pivotTime: Timestamp; // pivot bar closeTime
    pivotPrice: Price;
    confirmedAt: Timestamp; // pivotTime + 3 bars closeTime
    isConfirmed: boolean;
}

interface AnchorPoint {
    time: Timestamp; // pivot closeTime
    price: Price; // pivot price
}

interface Line2P {
    a: AnchorPoint; // older
    b: AnchorPoint; // newer
    slope: number; // (b.price - a.price) / (b.time - a.time)
    intercept: number; // price = slope*time + intercept
}

interface ChannelGeometry {
    dir: ChannelDir; // UP/DOWN
    anchorLine: Line2P; // UP: lower anchor line / DOWN: upper anchor line
    offset: number; // TF별 percentile
    midOffset: number; // offset*0.5
}

interface ChannelModel {
    id: string; // `${symbol}:${type}:${anchorStartTime}:${dir}:${offsetPctl}`
    symbol: string;
    type: ChannelType;
    tf: "D1" | "H4" | "H1" | "M30";
}

```

```

state: ChannelState;    // ACTIVE/INACTIVE/NONE
mode: ChannelMode;      // D1/H4=ENABLED 고정. H1/M30은 ENABLED or CONTEXT_ONLY

geometry?: ChannelGeometry;

anchorStartTime?: Timestamp; // a.time
anchorEndTime?: Timestamp;   // b.time
createdAt: Timestamp;        // 생성 시각(=해당 TF closeTime)
lastUpdatedAt: Timestamp;    // 마지막 업데이트(=해당 TF closeTime)

maxForwardBars: number;      // 300
displayUntil?: Timestamp;    // ACTIVE에서만

endTime?: Timestamp;
invalidReason?: ChannelInvalidReason;

ttlBars?: number;            // H1=100, M30=80
ttlStartTime?: Timestamp;    // ACTIVE 진입 시각(또는 생성 시각)
}

type PoiTrigger = "sweepRec" | "structure" | "disp";

interface ChannelPoi {
  id: string; // `${symbol}:CH_POI:${tf}:${time}:${dir}:${boundaryPrice}`
  symbol: string;
  tf: "D1" | "H4" | "H1" | "M30";
  dir: DIR; // BULL/BEAR (UP->BULL, DOWN->BEAR)
  createdAt: Timestamp; // 확정 시각(=해당 TF closeTime)
  boundaryPrice: Price; // POI 기준 경계 가격(해당 시점 boundaryLinePrice)
  triggers: PoiTrigger[];
  state: "ACTIVE" | "INACTIVE";
  endTime?: Timestamp;
  invalidReason?: ChannelInvalidReason | "expired_forward";
  dayKeyUtc?: string; // "YYYY-MM-DD"
}

```

3.3 상태머신 요약(전이/가드/이벤트)

3.1 CHANNEL 상태머신 (D1/H4/H1/M30 공통)

(없음)/NONE -> ACTIVE

- 입력: TF_BAR_CLOSE(now)
- 가드: 앵커 선택 성공(HL/LH) + residual 표본>=5
- 액션: ChannelModel 생성(state=ACTIVE), geometry 계산(offset percentile TF별), displayUntil 계산
- 이벤트: [CHANNEL][NEW][TF][UP|DOWN] time=...

ACTIVE -> INACTIVE (break)

- 입력: TF_BAR_CLOSE(now)
- 가드: Break(boundary) 규칙 TF별 충족
- 액션: endTime=now, invalidReason=break
- 이벤트: [CHANNEL][INVALID] reason=break ...

ACTIVE -> INACTIVE (anchor_invalid)

- 입력: TF_BAR_CLOSE(now)
- 가드: AnchorInvalid(anchor line) 규칙 TF별 충족
- 액션: endTime=now, invalidReason=anchor_invalid
- 이벤트: [CHANNEL][INVALID] reason=anchor_invalid ...

ACTIVE -> INACTIVE (ttl_expired) [H1/M30만]

- 입력: H1/M30_BAR_CLOSE(now)
- 가드: TTL 초과 (H1=100 bars, M30=80 bars)
- 액션: endTime=now, invalidReason=ttl_expired
- 이벤트: [CHANNEL][INVALID] reason=ttl_expired ...

ACTIVE -> INACTIVE (parent_poi_ended) [H1/M30 ENABLED만]

- 입력: H1/M30_BAR_CLOSE(now)
- 가드: 참조 상위 POI가 INACTIVE/DELETED로 전환
- 액션: endTime=now, invalidReason=parent_poi_ended
- 이벤트: [CHANNEL][INVALID] reason=parent_poi_ended ...

INACTIVE -> ACTIVE

- 입력: TF_BAR_CLOSE(now)
- 가드: 앵커 선택 성공 + residual>=5
- 액션: 새 ChannelModel 생성(state=ACTIVE)
- 이벤트: [CHANNEL][NEW][TF][UP|DOWN] time=...

3.2 H1/M30 mode 필드 업데이트 (전이 아님)

- 입력: H1/M30_BAR_CLOSE(now)
- parent POI 존재(활성 >=1) -> mode=ENABLED
- parent POI 0개 -> mode=CONTEXT_ONLY

3.3 CHANNEL_POI 생성

D1/H4:

- 입력: TF_BAR_CLOSE(now)
- 가드: gate(TF별) 통과 AND triggers 중 2개 이상
- 액션: ChannelPoi 생성(state=ACTIVE)
- 이벤트: [CHANNEL][POI][TF][BULL|BEAR] time=...

H1/M30:

- 입력: TF_BAR_CLOSE(now)
- 가드: mode=ENABLED AND gate 통과 AND triggers 3/3 AND (parent near OR parent inside) AND day cap 미초과
- 액션: ChannelPoi 생성(state=ACTIVE), daily counter+1
- 이벤트: [CHANNEL][POI][TF][BULL|BEAR] time=...

3.4 계산 함수 정의(고정)

overlapRatio(A=[aB,aT], B=[bB,bT]):

```
overlapLen = max(0, min(aT,bT) - max(aB,bB))
overlapRatio = overlapLen / min(heightA, heightB)
```

break 판단:

```
n연속 증가 이탈 + 이탈폭 >= ATR*X
n, X는 TF별 상수 (D1/H4: 2 & 0.20, H1: 1 & 0.30, M30: 1 & 0.35)
```

POI gate:
 $\text{abs}(\text{wickExtreme} - \text{boundaryPrice}) \leq \text{ATR} * \text{gateCoef}$ (TF별)
 wickExtreme: UP->wickLow, DOWN->wickHigh

Sweep->Recovery:
 sweep: wickExtreme가 boundary를 넘어감(이탈)
 recovery: 다음 봉 close가 boundary 안쪽으로 복귀 (1봉 고정)

Structure:
 fractal pivotLen=3 확정 pivot 기반 BOS/CHOC
 채널 방향 정합(UP이면 상승 구조, DOWN이면 하락 구조)

Displacement:
 $\max(\text{body over 3 bars}) > \text{ATR} * 1.0$ OR $\sum(\text{body over 3 bars}) > \text{ATR} * 1.8$

parent near / inside:
 near: $\text{abs}(\text{wickExtreme} - \text{parentPoiBoundary}) \leq \text{ATR}(\text{H1/M30}) * 0.20$
 inside: $\text{overlapRatio}(\text{parentPoiZone}, \text{boundaryZoneProxy}) \geq 0.20$
 boundaryZoneProxy = [boundary - gate, boundary + gate]
 gate는 TF별 POI gate 계수(0.08/0.06)*ATR 적용

dayKeyUtc:
 UTC 기준 날짜 문자열 "YYYY-MM-DD"
 cap key = `\${symbol}:\${tf}:\${dayKeyUtc}`

3.5 Rule Coverage Matrix(요약)

Rule Coverage Matrix (정본 항목 -> 필드/함수/전이 매핑)

- TF별 onBarClose: onBarClose(tf) / TF close 독립
- pivot 확정: updatePivots() / Pivot.isConfirmed / BOS/CHOC 가드
- 앵커 선택: selectAnchors() / minSwing=ATR*0.25 / geometry.anchorLine
- residual 표본>=5: collectResiduals() / MIN_RESIDUAL_SAMPLES
- offset 퍼센타일 TF별: percentile() / p95,p90,p85,p80
- 브레이크 TF별: checkBreak() / (n연속 + ATR*X)
- 앵커무효화: checkAnchorInvalid() / anchorLine 기준
- TTL(H1/M30): checkTtl() / H1=100, M30=80
- parent 종료 연동: checkParentEnded() / reason=parent_poi_ended
- mode 분류: computeMode() / ENABLED vs CONTEXT_ONLY
- POI gate TF별: checkPoiGate() / 0.15,0.12,0.08,0.06
- POI 트리거: detectTriggers() / sweepRec, structure, disp
- POI 확정: confirmPoi() / D1/H4 2of3, H1/M30 3of3
- H1/M30 추가 제한: checkParentNearInside(), capCounter
- UTC day cap: getDayKeyUtc() / 2개/일/심볼/TF
- 이벤트 포맷: formatIsoUtc(), emitChannelNew(), emitChannelInvalid(), emitChannelPoi(), (emitChannelMode optional)